

团队信息

项目	信息
团队名称	取名困难户
A 榜排名	第 10 名
A 榜最优总分	83.58
联系人	周若淼
微信号	zrm15006525987
联系电话	15006525987

项目概述

核心任务

本项目面向基于深度学习的点云降噪任务：输入受噪声污染的三维点云，由模型预测各点的位移向量，将偏离真实物体表面的点校正至表面附近，从而生成降噪点云。模型以 Chamfer Distance (CD) 和 Point-to-Surface Distance (P2S) 为主要评价指标，在降低噪声的同时尽可能保留物体的尖锐边缘与局部几何细节，并提升对不同噪声水平和未知物体类别的泛化能力。

解决思路

本项目采用两阶段级联的逐点位移预测方案：第一阶段模型对原始含噪点云进行初步降噪，预测点的位移并得到中间点云；第二阶段以该中间点云为输入，再次预测残余位移，对初步降噪结果进行进一步细化与校正，最终生成降噪点云。

模型选型

本项目选用 [PGD](#) (Guiding Point Cloud Denoising with Learned Structural Priors, AAAI 2026) 作为基础降噪模型。PGD 是一种单步点云降噪模型，通过预测各点的位移向量完成表面校正；模型引入向量量化码本，将输入特征中的代表性局部几何模式提炼为稳定的结构先验，并以此指导特征细化，使网络更准确地区分真实几何细节与噪声，因此适合作为本项目两阶段级联降噪框架中每个阶段的基础网络。

算法创新点

- 引入 NAA Encoder。** 在 PGD 的特征提取阶段，以基于邻域注意力的 [NAA Encoder](#) 替换原有局部特征提取模块。该模块通过 KNN 构建点的局部邻域，结合邻居特征与相对位置编码计算注意力权重，从而增强模型对复杂曲面和局部几何细节的表达能力。
- 采用两步式级联训练策略。** 首先训练单阶段 NAA-PGD，获得基础降噪权重；随后加载该权重构建 `noisy → PGD1 → PGD2 → denoised` 的两阶段级联模型，并继续微调。该策略在保留第一阶段稳定去噪能力的基础上，利用第二阶段进一步修正残余噪声与局部偏差。
- 加入 Hard-tail Loss。** 在级联训练中，计算每个预测点到 GT 点云最近点的平方距离，选取误差最大的部分点（本项目取 Top 10%）并求均值，作为额外损失项以较小权重加入总损失。该损失使模型在优化整体误差的同时重点关注困难点，从而减少局部残留噪声、离群点以及复杂区域中的较大偏差。

代码结构说明

项目代码位于根目录的 `code/` 下，按训练流程划分为三个阶段：`stage1` 训练单阶段模型；`stage2` 加载并冻结 PGD1，仅训练 PGD2；`stage3` 分别加载前两阶段权重，对两个模型进行联合微调，并按 `noisy → PGD1 → PGD2 → denoised` 完成级联预测。

```
code/
├─ stage1/
│   │   └─ configs/
│   │       │   └─ data/train.yaml
│   │       │   └─ model/core.yaml
│   │       │   └─ system/run.yaml
│   │       │   └─ task/train.yaml
│   │       └─ transform/ops.yaml
│   └─ datalist/
│       │   └─ train.txt
│       │   └─ validate.txt
│       └─ test.txt
│   └─ run.py
├─ stage2/
│   │   └─ configs/
│   │       │   └─ data/train.yaml
│   │       │   └─ model/core.yaml
│   │       │   └─ system/run.yaml
│   │       │   └─ task/freeze40.yaml
│   │       └─ transform/ops.yaml
│   └─ datalist/
│       └─ run.py
├─ stage3/
│   │   └─ configs/
│   │       │   └─ data/train.yaml
│   │       │   └─ data/predict.yaml
│   │       │   └─ model/core.yaml
│   │       │   └─ system/run.yaml
│   │       │   └─ task/joint.yaml
│   │       │   └─ task/predict.yaml
│   │       └─ transform/ops.yaml
│   └─ datalist/
│       └─ run.py
├─ stage*/src/
│   │   └─ data/
│   │       │   └─ asset.py
│   │       │   └─ datapath.py
│   │       │   └─ dataset.py
│   │       │   └─ augment.py
│   │       │   └─ transform.py
│   │       │   └─ spec.py
│   │       └─ utils.py
│   └─ model/
│       │   └─ pgd.py
│       │   └─ feature.py
│       │   └─ blocks.py
│       │   └─ infocd.py
│       │   └─ ops.py
│       │   └─ parse.py
│       └─ spec.py
# 训练单阶段 NAA-PGD
# 训练数据路径与 DataLoader 参数
# NAA-PGD 模型结构与去噪参数
# checkpoint 保存路径与命名
# stage1 训练任务入口配置
# 采样、加噪、增强及 patch 切分
# 训练样本列表
# 验证样本列表
# 测试样本列表
# stage1 训练入口
# 冻结 PGD1，训练 PGD2
# 第二阶段训练数据配置
# PGD2 模型配置
# PGD2 checkpoint 输出配置
# PGD1 权重及冻结训练参数
# 第二阶段数据处理流程
# 训练、验证与测试样本列表
# stage2 训练入口
# 两阶段联合微调与级联预测
# 联合训练数据配置
# 测试集参数配置
# 两阶段共用的 PGDModel 配置
# 合并 checkpoint 输出配置
# PGD1、PGD2 联合微调配置
# 两阶段预测配置
# 联合训练与预测预处理流程
# 训练与测试样本列表
# 联合训练、预测及权重拆分/合并入口
# 三个阶段均包含以下核心源码
# 点云与网格样本结构
# datalist 解析及 OBJ/NPY 懒加载
# Jitter Dataset 与 DataLoader
# 采样、归一化、加噪及几何增强
# 按 YAML 组合数据变换
# 数据配置基类
# 数据采样与几何辅助函数
# PGDModel 训练、验证与位移预测
# NAA 编解码器、码本及特征融合网络
# NAA、采样、上采样与 Codebook 模块
# InfoCD 类主损失
# Chamfer、KNN 与 patch 去噪算子
# 根据配置创建 PGDModel
# 模型配置与预测状态基类
```

```
|   └─ system/
|   └─ spec.py           # 训练循环与 checkpoint 保存
|   └─ pgd.py           # PGD 训练系统及日志输出
|   └─ parse.py         # 创建 system 与 writer
|   └─ utils/pointops.py # FPS、KNN、batch gather 等基础算子
└─ stage*/third_party/PointCloudLib/
    └─ misc/ops.py      # NAA Encoder 使用的 KNN/grouping 后端
```

其中，`stage*` 表示 `stage1`、`stage2` 和 `stage3` 中均包含的同名目录或文件。为避免重复，三个阶段共有的代码结构仅统一介绍一次。

环境配置步骤

基础环境

本项目使用以下环境完成训练与推理：

环境项	版本或要求
操作系统	Ubuntu 22.04
Python	3.9
Jittor	1.3.11.0, 满足 <code>Jittor >= 1.3.10</code>
NVIDIA 驱动	需兼容 CUDA 12.2 及以上版本
CUDA Toolkit	推荐使用 CUDA 12.4

项目所需 Python 依赖已统一记录在根目录的 `requirements.txt` 中。

安装依赖

建议使用 Conda 创建独立环境，并在项目根目录依次执行：

```
conda create -n pgd_jt python=3.9 -y
conda activate pgd_jt
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

配置系统 CUDA（可选）

如果机器已安装 CUDA 12.4，并希望 Jittor 调用系统 CUDA，可在运行前设置：

```
export PATH=/usr/local/cuda-12.4/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-12.4/lib64:$LD_LIBRARY_PATH
```

如果未配置系统 CUDA，Jittor 在首次运行时可能会自动下载所需组件，或调用其已有的 CUDA 缓存。首次编译所需时间通常会长于后续运行，请确保相关缓存目录具有可用空间和写入权限。

运行步骤

1. 配置数据集路径

运行前，分别修改三个阶段的训练数据配置：

```
code/stage1/configs/data/train.yaml  
code/stage2/configs/data/train.yaml  
code/stage3/configs/data/train.yaml
```

将其中的 `input_dataset_dir` 设置为实际训练集路径，例如：

```
input_dataset_dir: /path/to/dataset_train
```

随后修改预测数据配置 `code/stage3/configs/data/predict.yaml`，将测试集路径设置为：

```
input_dataset_dir: /path/to/dataset_test_noisy
```

2. 训练与预测

进入代码根目录并激活前文创建的环境：

```
cd code  
conda activate pgd_jt
```

第一步：训练单阶段 NAA-PGD

```
python stage1/run.py --task stage1/configs/task/train --seed 2025
```

checkpoint 默认保存在 `stage1/experiments/` 下。

第二步：冻结 PGD1，训练 PGD2

```
python stage2/run.py --task stage2/configs/task/freeze40
```

该阶段默认加载 `stage1/experiments/stage1_checkpoint_349.pkl`，输出 checkpoint 保存至 `stage2/experiments/`。

第三步：两阶段联合微调

```
python stage3/run.py --task stage3/configs/task/joint
```

该阶段默认加载以下权重：

```
stage1/experiments/stage1_checkpoint_349.pkl  
stage2/experiments/stage2_checkpoint_39.pkl
```

联合微调得到的 checkpoint 保存至 `stage3/experiments/`。

第四步：生成测试集降噪结果

```
python stage3/run.py --task stage3/configs/task/predict
```

预测任务默认加载 `stage3/experiments/stage3_checkpoint_joint_200.pkl`，并将结果保存至 `stage3/result/`。

3. 生成提交文件

预测完成后，进入 `stage3/` 目录并将 `result/` 打包为 `result.zip`：

```
cd stage3
zip -r result.zip result
```

最终提交文件为 `result.zip`，压缩包内的目录结构应为：

```
result/shapenet/<class>/<shape_id>/denoised.npy
```

其中，每个 `denoised.npy` 均为模型生成的降噪点云文件。

其他补充

1. **训练进程异常处理。** 若某一阶段在训练过程中出现进程异常退出、卡死或数据加载失败，可将该阶段对应配置文件 `configs/data/train.yaml` 中的 `num_workers` 参数设置为 `0`，使数据加载在主进程中执行，然后重新启动该阶段的训练任务。